

CSE235 Week 7: Strings

We've introduced the `str` data type as a way to store text data. This week, we'll look at Python's string data type more in depth.

Escape Sequences

There are some special characters that cannot be represented by directly typing them in a string. These are instead represented by escape sequences, which start with a backslash. An example is a new line, which is represented by `\n`. While this is technically two characters that you have to type, in a string it represents the one character that inserts a new line. For example:

```
print("Hello\nWorld")
```

will print:

```
Hello
World
```

Here is a list of some common escape sequences:

Sequence	Description
<code>\t</code>	Tab character
<code>\n</code>	New line
<code>\r</code>	Carriage return
<code>\\</code>	Backslash character
<code>\"</code>	Double-quote character
<code>\'</code>	Singe-quote character

F-Strings

One of the most useful string features that Python offers are f-strings. The f is short for format, and it allows you to embed formatting and values within a string literal without needing to concatenate the strings. Consider the following Python snippet:

```
x = int(input("Enter the first number: "))
y = int(input("Enter the second number: "))
z = x + y
result_str = str(x) + " + " + str(y) + " = " + str(z)
print(result_str)
# or, directly with print:
print(x, "+", y, "=", z)
```

Both of these are somewhat difficult to read. The first example using string concatenation has the casts from integers to strings, and the plus signs between the string segments. The second uses the print function to output the data, but it is also hard to read since print automatically inserts spaces. This can potentially make it hard to tell what exactly is being printed. F-strings allow for values, variables, and expressions to be embedded directly into the string literal. F-strings are denoted by putting an "f" right before the first double quote.

```
x = int(input("Enter the first number: "))
y = int(input("Enter the second number: "))
z = x + y
result_str = f"{x} + {y} = {z}"
print(result_str)
# or, directly within print:
print(f"{x} + {y} = {z}")
```

f-strings use curly braces within the string to denote when values, variables, and expressions are being inserted into the string. Beyond allowing data to be inserted into the string, f-strings also allow the data to be formatted. For example, this code will format the `float` with two decimal places:

```
amount = 26.7523
print(f"Amount = ${amount:.2f}")
# prints: Amount = $26.75
```

The colon after the variable name indicates that formatting is being started. The .2f indicates that a decimal point with rounded to two places. .4f would round to 4 decimal places.

Other String Variations

Beyond f-strings, there are a few other string variants. An r-string is a raw string. Raw strings treat backslashes as normal characters instead of as the start of an escape sequence. b-strings are byte strings. Normal Python strings can hold any Unicode character. Byte strings can only store ASCII characters. While byte strings can be represented as strings, the individual characters are treated as integers between 0 and 255. When printing a byte string, Python will output the string prefixed with a b instead of the plain text.

```
raw_string = r"Hello\nWorld"
byte_string = b"Hello World"
print(raw_string)
print(byte_string)
```

Output:

```
Hello\nWorld
b'Hello World'
```

String Member Functions

String has several member functions that allow for common operations. Parameters in square brackets are optional. Note: this is not an exhaustive list.

Function Name	Description
<code>center(width, fillchar=' ')</code>	Returns a string with the calling string centered in the width characters
<code>count(substr[, start[, end]])</code>	Counts the number of occurrences of <code>substr</code> in the string

Function Name	Description
<code>encode(encoding='utf-8', errors='strict')</code>	Converts the string to the specified byte encoding
<code>endswith(suffix[, start[, end]])</code>	Returns <code>True</code> if the string ends with <code>suffix</code>
<code>expandtabs(tabsize=8)</code>	Replaces the tab character with <code>tabsize</code> spaces
<code>find(substr[, start[, end]])</code>	Returns starting index of <code>substr</code> in the string
<code>isalnum()</code>	Returns <code>True</code> if the string is all letters and numbers
<code>isalpha()</code>	Returns <code>True</code> if the string is all letters
<code>isdigit()</code>	Returns <code>True</code> if the string is all numbers
<code>islower()</code>	Returns <code>True</code> if the string is all lowercase
<code>isspace()</code>	Returns <code>True</code> if the string is all whitespace
<code>isupper()</code>	Returns <code>True</code> if the string is all uppercase
<code>lower()</code>	Converts string to lowercase
<code>replace(old, new[, count])</code>	Replaces all or <code>count</code> (if specified) instances of <code>old</code> with <code>new</code>
<code>split(sep=None, maxsplit=-1)</code>	Returns a list of strings. The calling string is split on the <code>sep</code> string.

Function Name	Description
<code>strip(chars=None)</code>	Removes any of <code>chars</code> from the beginning and end of the string
<code>upper()</code>	Converts string to uppercase

The official Python documentation has the full list of string member functions.

Here are some basic examples of each of the member functions:

```

text = "Python"
print(text.center(12, "-"))
# prints: '---Python---'

text = "banana"
print(text.count("an"))
# prints: 2

text = "café"
# encode returns a b-string
print(text.encode(encoding="utf-8"))
# prints: b'caf\xc3\xa9'

filename = "report.pdf"
print(filename.endswith(".pdf"))
# prints: True

text = "A\tB\tC"
print(text.expandtabs(tabsize=4))
# prints: 'A    B    C'

text = "Find the needle in the haystack."
# returns the index of the start of the substring
print(text.find("needle"))
# prints: 9

username = "Player123"
print(username.isalnum())
# prints: True

print("Player 123".isalnum())
# prints: False (because of the space)

word = "HelloWorld"
print(word.isalpha())

```

```

# prints: True

code = "12345"
print(code.isdigit())
# prints: True

text = "hello world!"
# spaces and punctuation are ignored
print(text.islower())
# prints: True

spaces = " \n\t "
# new lines and tabs count as whitespace
print(spaces.isspace())
# prints: True

shouting = "LOOK OUT!"
# spaces and punctuation are ignored
print(shouting.isupper())
# prints: True

text = "Data Science"
print(text.lower())
# prints: 'data science'

text = "one potato, two potato, three potato"
print(text.replace("potato", "apple", 2))
# prints: 'one apple, two apple, three potato'

csv_line = "apple,banana,cherry"
# the comma is not included in the output strings
print(csv_line.split(","))
# prints: ['apple', 'banana', 'cherry']

text = "xxx  clean this up  xxx"
# this removes any x's or spaces at the start
# and end of the string
print(text.strip("x "))
# prints: 'clean this up'

text = "turn it up"
print(text.upper())
# prints: 'TURN IT UP'

```

String Indexes and Slicing

Strings are like lists, except every element is a single character. Therefore, the same indexing and slicing that works for lists works for strings as well.

```
greeting = "Hello World"
assert greeting[0] == 'H'
assert greeting[:5] == "Hello"
assert greeting[6:] == "World"
assert greeting[-1] == "d"
```

Conclusion

This week we discussed strings. Strings are used to hold text data. First, we looked at escape sequences, which are used to represent new lines and tabs, which cannot be directly typed. Then, we discussed f-strings, which allow for variables and expressions to be embedded directly into a string. Finally, we looked at the member functions for manipulating strings.